Answers to Your Lecture Slip Questions

Lecture slip of October 22, 2026 (Permutations and Operator Combinations)

CSCI 235 – Software Design and Analysis II

Hunter College, Fall 2026

Questions from the Oct 22 slip

Contents (click an entry to jump)

1. Exit slips
2. What does printAll do?
3. Slip question 1: printAll
4. Slip question 3: ops

Your questions → answers: exit slips (§1) what `printAll` does (§2) trace of slip question 1 (§3) trace of slip question 3 (§4)

When Will the Exit Slips Be Graded?

They are not graded. The exit slips are used only to check attendance.

Nothing on a slip affects your grade, so a wrong or blank answer costs you nothing. The slips are for you: they show what you can and cannot yet trace. The questions you write on them are answered in these notes.

What Does `printAll` Do?

It prints every arrangement (permutation) of the letters of a string. For `"CAT"`: $3! = 6$ strings.

```
void printAll(std::string& s, int k) {
  if (k == static_cast<int>(s.size())) { // every position filled
    std::cout << s << "\n";
    return;
  }
  for (int i = k; i < static_cast<int>(s.size()); ++i) {
    std::swap(s[k], s[i]);              // choose
    printAll(s, k + 1);                 // explore
    std::swap(s[k], s[i]);              // undo
  }
}
```

Output for `"CAT"`: `CAT`, `CTA`, `ACT`, `ATC`, `TAC`, `TCA`.

Analogy: Filling Seats in a Row

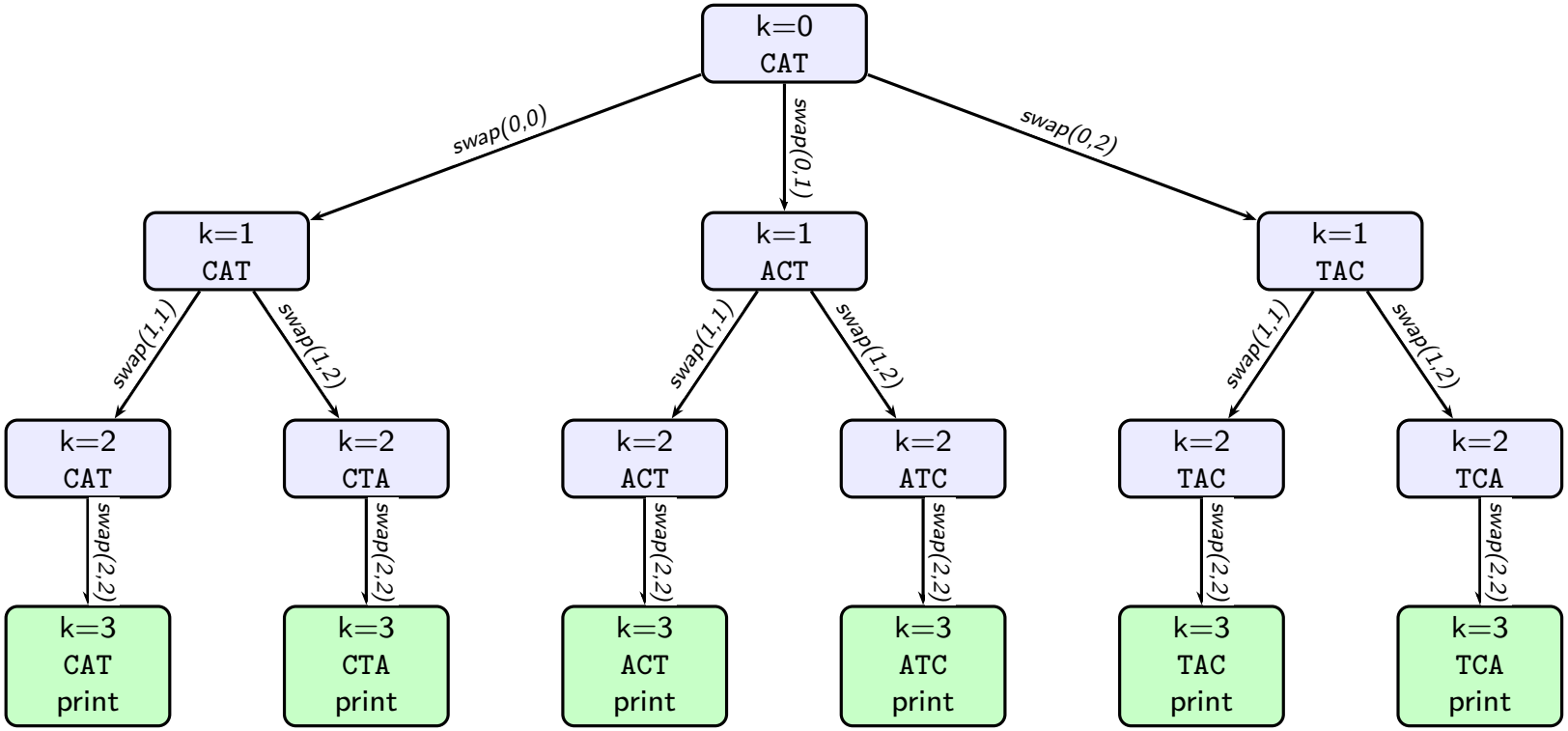
Positions are **seats**, letters are **people**. `printAll(s, k)` means: *seats* 0 *to* $k-1$ *are filled and stay fixed; now fill seat* `k`.

- **Who may sit in seat `k`?** Anyone not yet seated: the letters in positions `k` to the end. The loop over `i` tries each.
- **Choose:** `swap(s[k], s[i])` brings candidate `i` into seat `k` (if `i == k`, nothing changes).
- **Explore:** `printAll(s, k + 1)` fills the remaining seats in every way.
- **Undo:** swap back, so the next candidate starts from the same string.
- **Base case:** `k == s.size()`: all seats filled, print.

Why $n!$: seat 0 has n candidates, seat 1 has $n-1$, ..., so $n \times (n-1) \times \cdots \times 1 = n!$.

Call Tree of `printAll("CAT", 0)`

Each box: `k` and `s` when the call starts. Arrow label: the swap done just before the call. Green = prints. Left to right, depth first. $1 + 3 + 6 + 6 = 16$ calls.



(c) `printAll` is called with `k = 1` **3** times (once per `i` at the top).

Every Swap and Undo (1 of 3)

#	Call	Event	s after
1	k=0	choose: swap(0,0) (same), call k=1	CAT
2	k=1	choose: swap(1,1) (same), call k=2	CAT
3	k=2	choose: swap(2,2) (same), call k=3	CAT
4	k=3	**print**	CAT ←
5	k=2	undo: swap(2,2)	CAT
6	k=1	undo: swap(1,1)	CAT
7	k=1	choose: swap(1,2), call k=2	CTA
8	k=2	choose: swap(2,2) (same), call k=3	CTA
9	k=3	**print**	CTA ←
10	k=2	undo: swap(2,2)	CTA
11	k=1	undo: swap(1,2)	CAT
12	k=0	undo: swap(0,0)	CAT

Every Swap and Undo (2 of 3)

#	Call	Event	s after
13	k=0	choose: swap(0,1), call k=1	ACT
14	k=1	choose: swap(1,1) (same), call k=2	ACT
15	k=2	choose: swap(2,2) (same), call k=3	ACT
16	k=3	**print**	ACT ←
17	k=2	undo: swap(2,2)	ACT
18	k=1	undo: swap(1,1)	ACT
19	k=1	choose: swap(1,2), call k=2	ATC
20	k=2	choose: swap(2,2) (same), call k=3	ATC
21	k=3	**print**	ATC ←
22	k=2	undo: swap(2,2)	ATC
23	k=1	undo: swap(1,2)	ACT
24	k=0	undo: swap(0,1)	CAT

Every Swap and Undo (3 of 3)

#	Call	Event	s after
25	k=0	choose: swap(0,2), call k=1	TAC
26	k=1	choose: swap(1,1) (same), call k=2	TAC
27	k=2	choose: swap(2,2) (same), call k=3	TAC
28	k=3	**print**	TAC ←
29	k=2	undo: swap(2,2)	TAC
30	k=1	undo: swap(1,1)	TAC
31	k=1	choose: swap(1,2), call k=2	TCA
32	k=2	choose: swap(2,2) (same), call k=3	TCA
33	k=3	**print**	TCA ←
34	k=2	undo: swap(2,2)	TCA
35	k=1	undo: swap(1,2)	TAC
36	k=0	undo: swap(0,2)	CAT

Checking the Slip, and the Undo at Work

(a) Printed in order: `CAT`, `CTA`, `ACT`, `ATC`, `TAC`, `TCA`. **(b)** After the call returns, `s` is `"CAT"` again: every choose has a matching undo.

Watch rows 11 to 13. Rows 11 and 12 undo the swaps that produced `CTA`, so `s` is `CAT` again. Only then does row 13 do `swap(s[0], s[1])`, which brings `A` to the front: `ACT`.

If the undo in row 11 were missing, row 13 would swap letters of `CTA` and print a wrong string.

ops: What It Does

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
  if (k == static_cast<int>(v.size())) { std::cout << expr << "\n";
      return; }
  for (char op : std::string("+-*/")) {
    std::size_t len = expr.size();
    expr += op;
    expr += std::to_string(v[k]);       // choose
    ops(v, k + 1, expr);                // explore
    expr.resize(len);                   // undo
  }
}
// v = {4, 5, 6, 7}; std::string e = "4"; ops(v, 1, e);
```

Call	Gap filled	Calls
`ops(v, 1, expr)`	4 and 5	1
`ops(v, 2, expr)`	5 and 6	4
`ops(v, 3, expr)`	6 and 7	16
`ops(v, 4, expr)`	print	64

64 lines from 85 calls; the last gap changes fastest.

ops: Events 1 to 12

#	Call	Event	expr after
1	k=1	choose: append +5, call k=2	4+5
2	k=2	choose: append +6, call k=3	4+5+6
3	k=3	choose: append +7, call k=4	4+5+6+7
4	k=4	**print**	4+5+6+7 ←
5	k=3	undo: resize	4+5+6
6	k=3	choose: append -7, call k=4	4+5+6-7
7	k=4	**print**	4+5+6-7 ←
8	k=3	undo: resize	4+5+6
9	k=3	choose: append *7, call k=4	4+5+6*7
10	k=4	**print**	4+5+6*7 ←
11	k=3	undo: resize	4+5+6
12	k=3	choose: append /7, call k=4	4+5+6/7

ops: Events 13 to 24

#	Call	Event	expr after
13	k=4	**print**	4+5+6/7 ←
14	k=3	undo: resize	4+5+6
15	k=2	undo: resize	4+5
16	k=2	choose: append -6, call k=3	4+5-6
17	k=3	choose: append +7, call k=4	4+5-6+7
18	k=4	**print**	4+5-6+7 ←
19	k=3	undo: resize	4+5-6
20	k=3	choose: append -7, call k=4	4+5-6-7
21	k=4	**print**	4+5-6-7 ←
22	k=3	undo: resize	4+5-6
23	k=3	choose: append *7, call k=4	4+5-6*7
24	k=4	**print**	4+5-6*7 ←

Row 15: back to 4+5. Row 16: the - branch at gap 2 starts. Blocks of 4 lines differ only in the last gap.

Slip Question 3(b): Counting Expressions

$4! \times 4^3 \times 5 = 24 \times 64 \times 5 = \mathbf{7\,680}$.

- $4! = 24$: orders of the four numbers.
- $4^3 = 64$: operators for the three gaps.
- 5: parenthesizations of four operands.

Multiply, because each choice is independent. This counts *expressions*, not distinct values: $(4 + 5) \times (6 + 7)$ and $(5 + 4) \times (6 + 7)$ are two expressions with the same value.

All 64 Lines Printed by `ops(v, 1, e)`

first gap +	first gap −	first gap *	first gap /
1. 4+5+6+7	17. 4−5+6+7	33. 4*5+6+7	49. 4/5+6+7
2. 4+5+6−7	18. 4−5+6−7	34. 4*5+6−7	50. 4/5+6−7
3. 4+5+6*7	19. 4−5+6*7	35. 4*5+6*7	51. 4/5+6*7
4. 4+5+6/7	20. 4−5+6/7	36. 4*5+6/7	52. 4/5+6/7
5. 4+5−6+7	21. 4−5−6+7	37. 4*5−6+7	53. 4/5−6+7
6. 4+5−6−7	22. 4−5−6−7	38. 4*5−6−7	54. 4/5−6−7
7. 4+5−6*7	23. 4−5−6*7	39. 4*5−6*7	55. 4/5−6*7
8. 4+5−6/7	24. 4−5−6/7	40. 4*5−6/7	56. 4/5−6/7
9. 4+5*6+7	25. 4−5*6+7	41. 4*5*6+7	57. 4/5*6+7
10. 4+5*6−7	26. 4−5*6−7	42. 4*5*6−7	58. 4/5*6−7
11. 4+5*6*7	27. 4−5*6*7	43. 4*5*6*7	59. 4/5*6*7
12. 4+5*6/7	28. 4−5*6/7	44. 4*5*6/7	60. 4/5*6/7
13. 4+5/6+7	29. 4−5/6+7	45. 4*5/6+7	61. 4/5/6+7
14. 4+5/6−7	30. 4−5/6−7	46. 4*5/6−7	62. 4/5/6−7
15. 4+5/6*7	31. 4−5/6*7	47. 4*5/6*7	63. 4/5/6*7
16. 4+5/6/7	32. 4−5/6/7	48. 4*5/6/7	64. 4/5/6/7